



LeetCode 239 — Sliding Window Maximum

 <https://leetcode.com/problems/sliding-window-maximum/>

1. Problem Title & Link

Title: LeetCode 239: Sliding Window Maximum

Link: <https://leetcode.com/problems/sliding-window-maximum/>

2. Problem Statement — Simple Summary

Given an array `nums[]` and a window size `k`, slide the window from left to right and return an array of **maximum values in each window**.

Window size = `k`

Move window by 1 each time → output max

Goal: Compute each window maximum in **$O(n)$** .

3. Examples

Example 1

Input:

`nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

Output:

`[3,3,5,5,6,7]`

Explanation windows:

`[1,3,-1] → 3`

`[3,-1,-3] → 3`

`[-1,-3,5] → 5`

`[-3,5,3] → 5`

`[5,3,6] → 6`

`[3,6,7] → 7`

Example 2

`nums=[1]`, `k=1 → [1]`

4. Constraints

- $1 \leq n \leq 10^5$
- $1 \leq k \leq n$
- Must be **better than $O(n*k)$**
- Target: **$O(n)$ using Deque**

5. Core Concept



★ Monotonic Deque (Decreasing Order)

Deque stores *indices*, always in **decreasing order of values**.

→ Front always has the current maximum.

6. Thought Process (Step-by-Step)

✗ Brute Force (TLE)

For each window scan k elements → $O(n*k)$.

✓ Optimized — Monotonic Deque

Rules while sliding:

1. Remove indices from front if out of window ($i-k+1$)
2. Remove from back while $nums[back] < nums[current]$
3. Add current index
4. The front of deque is the max for current window

Why it works?

- Removing smaller values ensures deque always holds possible maximum candidates.
- Each element enters and leaves deque **at most once** → $O(n)$.

7. Visual Walkthrough

$nums = [1, 3, -1, -3, 5, 3, 6, 7]$, $k=3$

Deque holds **indices**, values in decreasing order:

i	num	deque values (desc)	window max
0	1	[1]	-
1	3	[3] ← 1 removed	-
2	-1	[3, -1]	3
3	-3	[3, -1, -3]	3
4	5	[5] ← removed 3, -1, -3	5
5	3	[5, 3]	5
6	6	[6] ← removed 5, 3	6
7	7	[7] ← removed 6	7

Result → [3, 3, 5, 5, 6, 7]

8. Pseudocode

```

deque = empty
result = []

for i in range(n):
    if deque.front out of window (i-k+1):

```



```
        pop front

while deque not empty and nums[deque.back] < nums[i]:
    pop back

push i into deque

if i >= k-1:
    result.append(nums[deque.front])

return result
```

9. Code Implementation

Python — $O(n)$

```
from collections import deque

class Solution:
    def maxSlidingWindow(self, nums, k):
        dq = deque()
        res = []

        for i in range(len(nums)):
            # Remove out-of-window
            if dq and dq[0] <= i - k:
                dq.popleft()

            # Maintain decreasing deque
            while dq and nums[dq[-1]] < nums[i]:
                dq.pop()

            dq.append(i)

            # Start recording max from index >= k-1
            if i >= k - 1:
                res.append(nums[dq[0]])

        return res
```

Java — $O(n)$

```
import java.util.*;
```



```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        Deque<Integer> dq = new ArrayDeque<>();
        int n = nums.length;
        int[] res = new int[n - k + 1];
        int idx = 0;

        for (int i = 0; i < n; i++) {

            if (!dq.isEmpty() && dq.peekFirst() <= i - k)
                dq.pollFirst();

            while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i])
                dq.pollLast();

            dq.offerLast(i);

            if (i >= k - 1)
                res[idx++] = nums[dq.peekFirst()];
        }
        return res;
    }
}
```

10. Time & Space Complexity

Measure	Complexity
Time	O(n) — each index pushed & popped $\leq$ once
Space	O(k) deque max size

11. Common Mistakes / Edge Cases

- ✗ Storing values instead of indices
- ✗ Not removing old indices (window overflow)
- ✗ Not popping smaller values from back
- ✗ Incorrect start for result ($i \geq k-1$)
- ✓ Edge cases
 - $k = 1 \rightarrow$ return nums
 - All equal $\rightarrow$ deque holds k values



- Strictly increasing/ decreasing arrays

12. Dry Run Table

nums = [4,2,12,3,-1,5], k=2

i	dq(after)	max
0	[4]	-
1	[4,2]	4
2	[12]	12
3	[12,3]	12
4	[12,-1]	12
5	[12,5]	12

13. Use Cases (Real Life)

- Stock price sliding window max
- CPU/Memory monitoring over time
- Sensor peak readings
- Streaming window data analysis

14. Traps to Avoid

- ⚠ Using max heap → $O(n \log k)$ slower
- ⚠ Recomputing max every time → $O(nk)$
- ⚠ Forgetting index-based window bounds

15. Builds To Related Problems

Problem	Relation
LC 316 / LC 1081	Monotonic stack variations
LC 901 Stock Span	Opposite monotonic idea
LC 862 / 1696	Sliding window with monotonic queue
LC 1871	Windows with BFS/DP

16. Alternate Approaches

Approach	Time	When to Use
Monotonic Deque ✓	$O(n)$	Best — standard
Max Heap	$O(n \log k)$	OK if n small
Block Preprocessing	$O(n)$	RMQ style — advanced



17. Why This Works (1-line Intuition)

The deque always keeps **potential maximums in order**, removing anyone weaker — so front is always the instant answer.

18. Follow-Up Explorations

- Return **min sliding window** (switch comparison)
- Track indices + values inside window
- Dynamic window size (variable k)
- 2D sliding window max (matrix)